

Segmentation



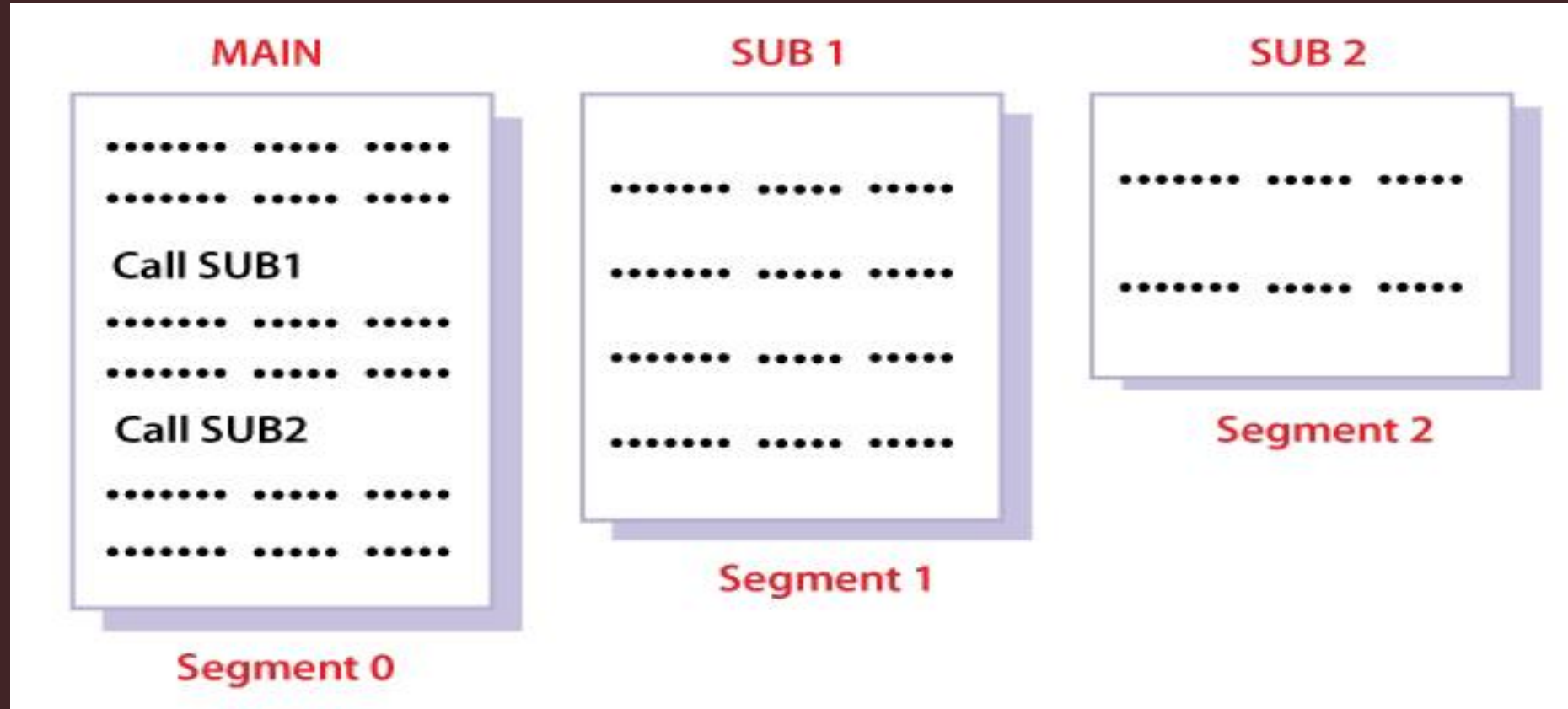
Segmentation *n*

- Segmentation is a memory management technique in which the memory is divided into the variable size parts. Each part is known as a segment which can be allocated to a process.
- Process is divided into different segment, that segment is loaded into memory.
- Details about each segment are stored in a table called a segment table.
- Segment table contains mainly two information about segment:
- Base: It is the base address of the segment
- Limit: It is the length of the segment.

Segmentation

- Paging is more closer to the Operating system rather than the User. It divides all the processes into the form of pages regardless of the fact that a process can have some relative parts of functions which need to be loaded in the same page.
- Segmentation which divides the process into the segments.
- Each segment contains the same type of functions such as the main function can be included in one segment and the library functions can be included in the other segment.

Segmentation

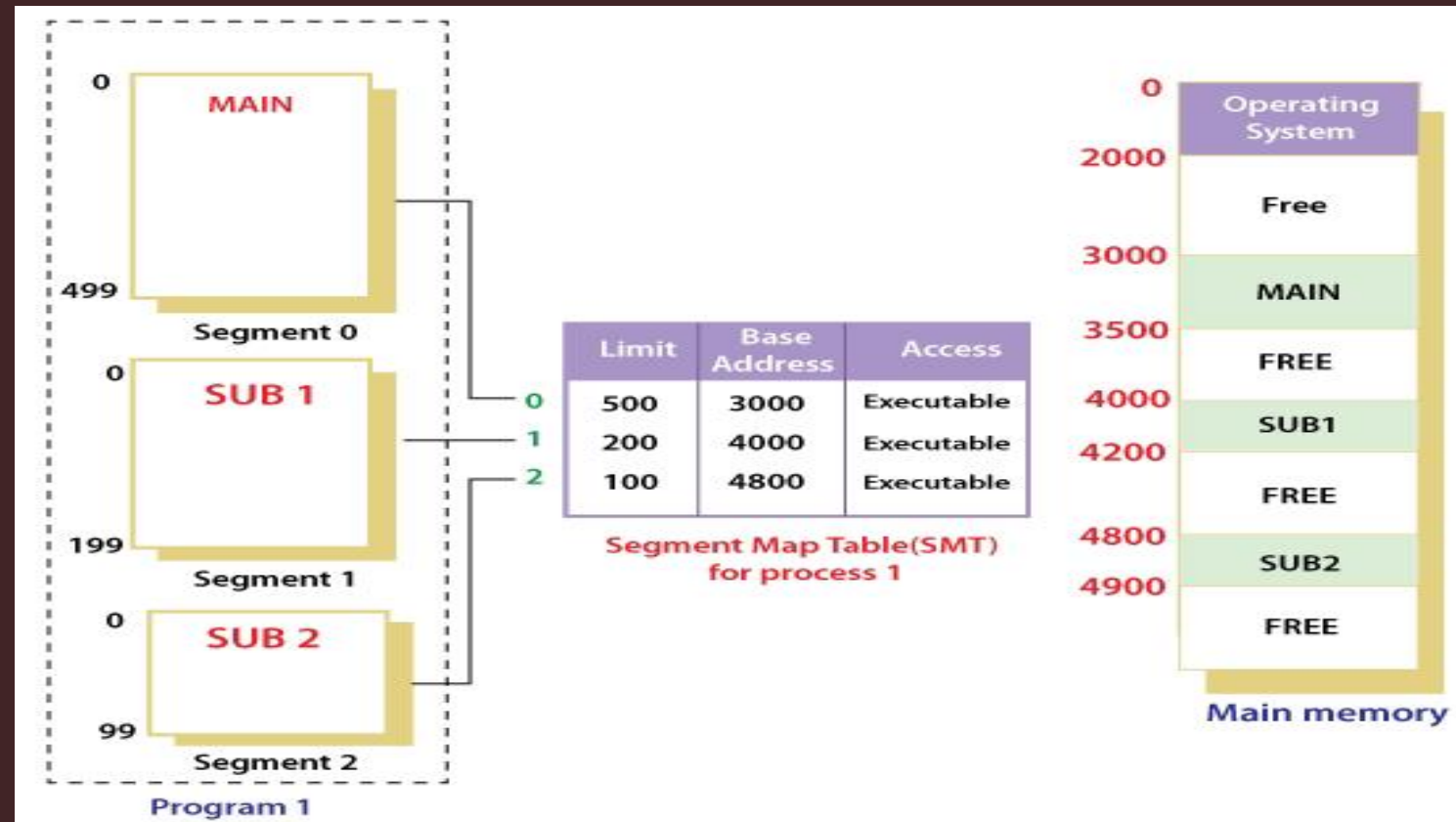


Segmentation-Address Generation

- CPU generates a logical address which contains two parts:
- 1) Segment Number
- 2) Offset

Segmentation

- When a program is loaded into memory, the segmentation system tries to locate space that is large enough to hold the first segment of the process, space information is obtained from the free list maintained by memory manager. Then it tries to locate space for other segments.



Segmentation Hardware

- Each entry of the segment table has a segment base and a segment limit. The segment base contains the starting physical address where the segment resides in memory, whereas the segment limit specifies the length of the segment.
- A logical address consists of two parts: a segment number, s , and an offset into that segment, d .
- The segment number is used as an index into the segment table. The offset d of the logical address must be between 0 and the segment limit. If it is not, we trap to the operating system (logical addressing attempt beyond end of segment).
- If this offset is legal, it is added to the segment base to produce the address in physical memory of the desired byte. The segment table is thus essentially an array of base-limit register pairs

Segmentation Hardware

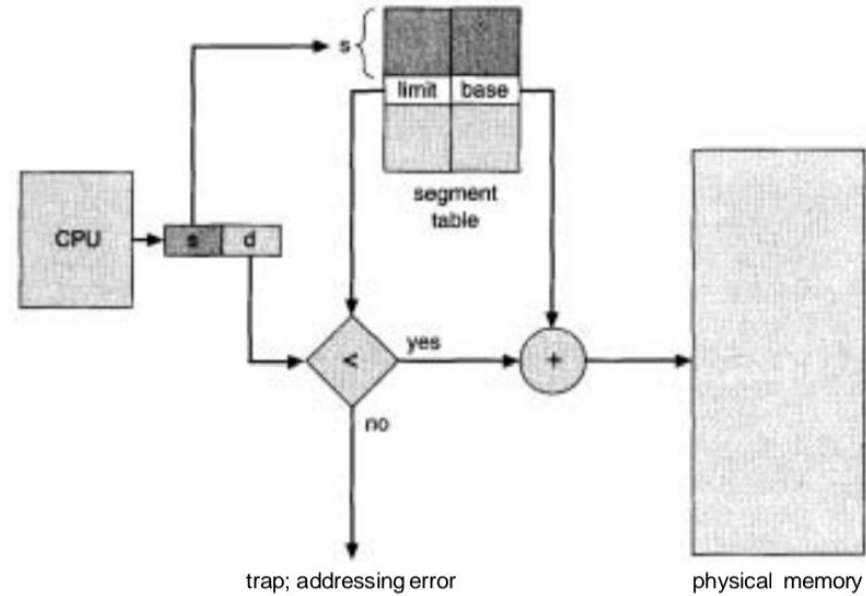


Figure 9.18 Segmentation hardware.

Segmentation

Segment number is mapped to the segment table. The limit of the respective segment is compared with the offset. If the offset is less than the limit then the address is valid otherwise it throws an error as the address is invalid.

→ In the case of valid addresses, the base address of the segment is added to the offset to get the physical address of the actual word in the main memory.

Segmentation with paging (paged segmentation)

- A program is divided into segments (like code, data, stack, etc.).
- Each segment is further divided into pages of equal size.
- Physical memory is divided into frames, just like in paging.
- Pages are smaller than segments.
- Each Segment has a page table which means every program has multiple page tables.
- The logical address is represented as Segment Number (base address), Page number and page offset.

Segmentation with paging

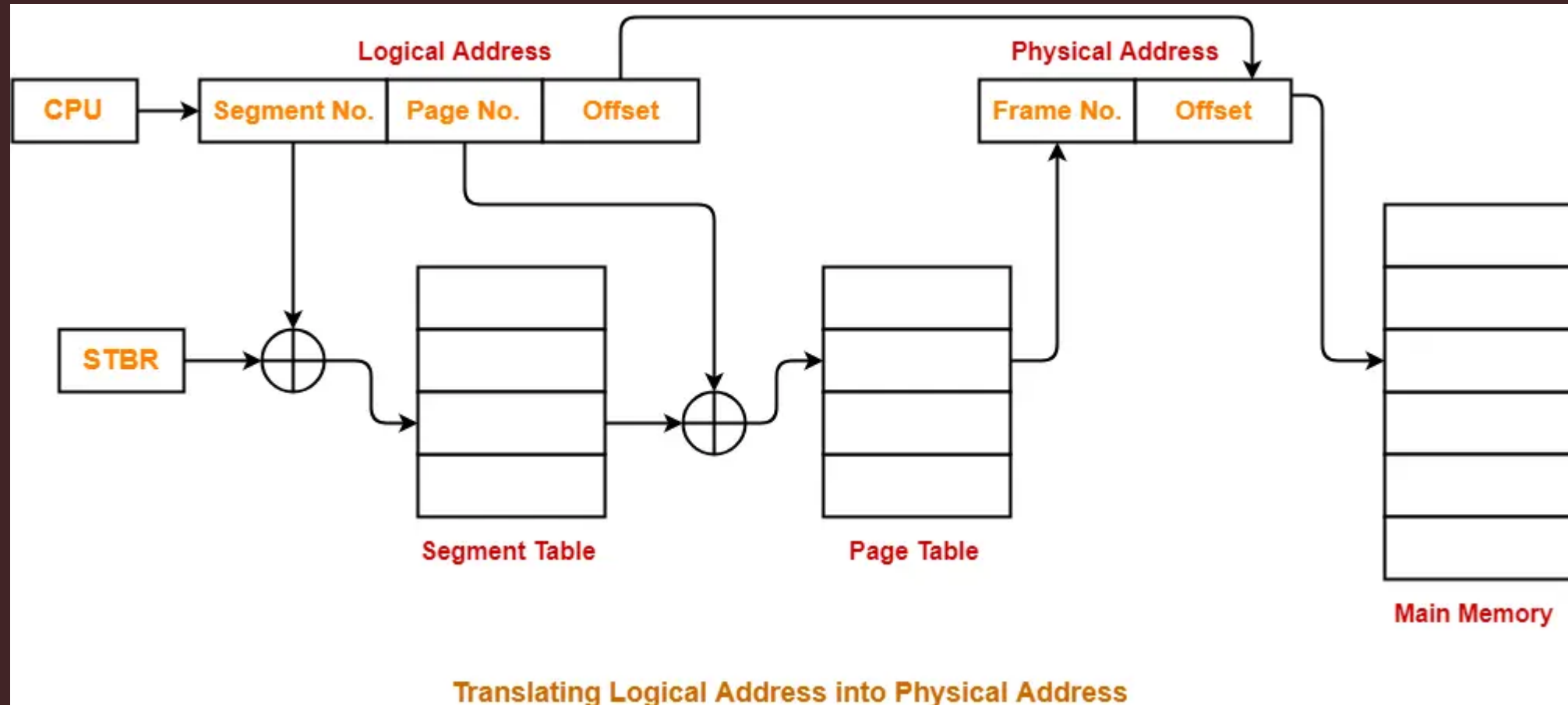
Segment Number → It points to the appropriate Segment Number.

Page Number → It Points to the exact page within the segment

Page Offset → Used as an offset within the page frame

Segment	Description	Pages	Has page table?
0	Code	8 pages	One page table (8 entries)
1	Data	4 pages	One page table (4 entries)
2	Stack	2 pages	One page table (2 entries)

Segmentation with paging



- **How is a logical address translated to a physical address?**
- **Segment number (s)** → used to index into the **segment table**.
 - Each segment table entry stores:
 - **Base address** (starting address of that segment's page table).
 - **Limit** (number of pages in the segment).
- **Page number (p)** → used to index into that segment's **page table**.
 - Each entry gives the **frame number** in physical memory.
- **Offset (d)** → added to the **frame's base address** to get the **final physical address**.
- *The **page table's job** is to map **each page of that segment** to physical frames.*
- *Program → divided into → Segments → each segment divided into → Pages → mapped to → Frames*